



ble-sensors-mqtt

Manuale di installazione e utilizzo

Version 1.0.0 - Edizione italiana

Alessandro De Salvo - EUPL-1.2

1. Scopo

ble-sensors-mqtt 1.0.0 è un gateway multi-sensore per Raspberry Pi. I plugin acquisiscono dati BLE o Tuya Cloud e producono un formato comune con identificatore, nome, marca, modello, protocollo, segnale, timestamp e tutti i valori disponibili. Lo stesso snapshot alimenta MQTT, Prometheus e SNMP.

2. Installazione

```
sudo apt update
sudo apt install -y bluetooth bluez python3 python3-venv
sudo systemctl enable --now bluetooth
python3 -m venv .venv
.venv/bin/pip install --upgrade pip
.venv/bin/pip install '.*[all]'
```

Sul Pi Zero usare `pip install .` per il solo SwitchBot, `.[sensors]` per i decoder BLE o `.[cloud]` per SwitchBot e Tuya. Verificare con `ble-sensors-mqtt --list-plugins`.

3. Plugin supportati

Nome	Famiglia	Accesso
switchbot	SwitchBot	BLE locale
xiaomi	Xiaomi/Mijia/HHCC	BLE, eventuale bind key
govee	Govee	BLE locale
inkbird	Inkbird	BLE/connezione su modelli compatibili
thermopro	ThermoPro	BLE locale
qingping	Qingping/ClearGrass	BLE locale
bthome	BTHome, Shelly BLU, ATC/PVX-BTHome	BLE locale
ruuvi	RuuviTag	BLE locale
sensornpush	SensorPush	BLE locale
airthings	Airthings	BLE GATT attivo
mopeka	Mopeka	BLE locale
tuya-cloud	Tuya/Smart Life	HTTPS cloud

Il supporto dipende da modello e firmware.

4. Scan, selezione e alias

```
.venv/bin/ble-sensors-mqtt --scan --scan-duration 15
.venv/bin/ble-sensors-mqtt --scan --plugin switchbot --plugin bthome
.venv/bin/ble-sensors-mqtt --mqtt-host 192.0.2.10 \
--device AA:BB:CC:DD:EE:FF \
--device-name 'AA:BB:CC:DD:EE:FF=Sala Server'
```

Lo scan non richiede MQTT. Per Tuya o sotto-canali usare `--sensor-name 'ID=NOME'`. L'alias è propagato a ogni output e il nome BLE originale resta in `bluetooth_name`.

Per Xiaomi/BTHome cifrati, salvare la bind key esadecimale in un file protetto e aggiungere al TOML: `[ble_keys."MAC"], plugin="xiaomi" o "bthome", e bindkey_file="/percorso/file"`. Non passare mai la chiave in CLI.

5. Esecuzione e MQTT

Home Assistant MQTT Discovery

Aggiungere `--home-assistant-discovery` per pubblicare la configurazione retained di Home Assistant MQTT Discovery. Il prefisso discovery predefinito è `homeassistant`; può essere cambiato con

`--home-assistant-discovery-prefix`. Ogni sensore esportato diventa un device Home Assistant, con una entità sensor per ogni valore scalare del payload più le entità diagnostiche RSSI e protocollo. Lo stato resta sul normale topic MQTT del gateway e l'availability è legata a `<mqtt-prefix>/bridge/status`.

```
.venv/bin/ble-sensors-mqtt --mqtt-host 127.0.0.1 --home-assistant-discovery
```

La configurazione discovery è retained. I topic discovery obsoleti vengono rimossi esplicitamente con payload retained vuoti, così sensori/entità rimossi scompaiono da Home Assistant. Nei deployment persistenti/systemd mantenere attivo il file di stato runtime per consentire il cleanup anche dopo i riavvii.

```
.venv/bin/ble-sensors-mqtt --mqtt-host 127.0.0.1 --once
.venv/bin/ble-sensors-mqtt --mqtt-host 192.0.2.10 --poll-interval 60 --allow-insecure-mqtt
.venv/bin/ble-sensors-mqtt --mqtt-host mqtt.example.it --mqtt-port 8883 \
  --mqtt-username sensor-publisher \
  --mqtt-password-file /etc/ble-sensors-mqtt/mqtt-password --mqtt-tls
```

Default: polling ogni 30 secondi. Il certificato TLS è sempre verificato. I file segreti devono essere 0640 o più restrittivi. Ogni payload contiene `manufacturer`, `model`, `protocol` e, in `data`, tutte le informazioni del decoder.

6. Tuya Cloud

Usare `--cloud-help`, creare un progetto Smart Home su Tuya IoT e collegare l'account Smart Life/Tuya. Conservare Access ID e Access Secret in file distinti protetti.

```
[cloud.tuya]
enabled = true
region = "eu"
api_key_file = "/etc/ble-sensors-mqtt/tuya-api-key"
api_secret_file = "/etc/ble-sensors-mqtt/tuya-api-secret"
api_device_id = "DEVICE_ID_DI_RIFERIMENTO"
device_ids = ["DEVICE_ID_1"]
```

Il TOML deve essere 0640 o più restrittivo. Provare con:

```
.venv/bin/ble-sensors-mqtt --scan --cloud-config /etc/ble-sensors-mqtt/cloud.toml
```

Un guasto Tuya non interrompe la raccolta BLE. Valutare privacy e dipendenza dal cloud. I dispositivi Tuya con firmware BTHome possono funzionare localmente.

7. Prometheus

```
.venv/bin/ble-sensors-mqtt --mqtt-host 192.0.2.10 --prometheus
curl http://127.0.0.1:9105/metrics
```

Ogni serie ha label indirizzo, nome, marca, modello e protocollo. Le metriche storiche per temperatura, umidità, batteria e RSSI restano disponibili. Ogni altro numero/booleano usa `ble_sensors_sensor_value`; le stringhe usano `ble_sensors_sensor_info`.

```
--prometheus --prometheus-host 0.0.0.0 --prometheus-port 9200 --allow-external-prometheus
```

Default 127.0.0.1:9105. 0.0.0.0/: : abilita accesso esterno se il firewall lo consente. L'endpoint non ha autenticazione/TLS: usare VPN, reverse proxy o ACL di rete.

8. SNMP

```
.venv/bin/ble-sensors-mqtt --mqtt-host 192.0.2.10 --snmp \
  --snmp-community-file /etc/ble-sensors-mqtt/snmp-community
```

Default 127.0.0.1:1161. Usare `--snmp-host 0.0.0.0 --allow-external-snmp/: :` e `--snmp-port` per accesso esterno. La tabella .10 include marca, modello e protocollo; .20 espone ogni scalare come chiave, valore, tipo e unità. SNMPv2c non cifra: limitare UDP o usare VPN/proxy SNMPv3.

9. systemd

Per una installazione completa a partire dal clone GitHub (prerequisiti host, controllo Bluetooth, venv e systemd):

```
git clone https://github.com/desalvo/ble-sensors-mqtt.git
cd ble-sensors-mqtt
sudo scripts/install-from-github.sh --mqtt-host mqtt.example.net --prometheus
```

Usare l'installer systemd direttamente quando il clone e i prerequisiti sono già presenti:

```
# Interattivo: i valori CLI sono i default mostrati nelle domande
sudo scripts/install-systemd.sh --mqtt-host mqtt.example.net --prometheus

# Provisioning completamente non interattivo
sudo scripts/install-systemd.sh --non-interactive \
  --mqtt-host mqtt.example.net --mqtt-tls \
  --mqtt-username ble-sensors-publisher --mqtt-password-file ./mqtt-password \
  --home-assistant-discovery --prometheus
```

L'installer crea utente di servizio, venv isolata, configurazione root-owned e /etc/ble-sensors-mqtt/service-args.json; argomenti ripetuti e valori con spazi sono preservati senza parsing shell. In modalità interattiva i valori CLI restano i default mostrati dal wizard e le opzioni ripetibili già passate vengono mantenute. Con --non-interactive l'installazione usa esclusivamente valori CLI/default. Vedere docs/SYSTEMD.it.md.

10. Docker e Kubernetes

```
# Build multiarch + push Docker Hub di desalvo/ble-sensors-mqtt:<VERSION>
docker login
scripts/build-docker.sh

# Demone continuo Docker
cd docker
cp .env.example .env
cp arguments.example arguments
docker compose up -d

# Demone continuo Kubernetes
kubectl label node NOME_NODO ble-sensors-mqtt/bluetooth=true
kubectl apply -k kubernetes/
```

Prima di Docker/Kubernetes verificare l'host con `sudo scripts/check-bluetooth-host.sh --strict`. Entrambi montano /run/dbus read-only e usano BlueZ dell'host via system D-Bus; non richiedono modalità privilegiata nel modello supportato. Kubernetes include anche kubernetes/bluetooth-test-pod.yaml per validare --scan sul nodo.

L'immagine supporta linux/amd64 e linux/arm64. Docker/Kubernetes montano il system D-Bus host per BlueZ, persistono lo stato runtime, inviano MQTT in uscita ed espongono Prometheus TCP/9105 e SNMP UDP/1161. Manifest LoadBalancer Kubernetes opzionali espongono Prometheus e SNMP all'esterno solo se applicati esplicitamente. scripts/build-docker.sh esegue di default il push di desalvo/ble-sensors-mqtt:<VERSION>; la CI può pubblicare l'immagine versionata sui tag vX.Y.Z e latest su main. Vedere docs/DOCKER.it.md e docs/KUBERNETES.it.md; gli endpoint di monitoring esterni vanno limitati a reti fidate.

11. Estensioni

Un plugin esterno implementa SensorPlugin.decode() e restituisce SensorReading; si registra nel gruppo entry point ble_sensors_mqtt.sensor_plugins. Ogni lettura valorizza marca, modello e protocollo. Le eccezioni dei plugin sono isolate.

12. Sicurezza e diagnosi

- non eseguire come root;
- proteggere file MQTT, Tuya e SNMP;
- usare ACL MQTT minime e TLS;
- non esporre Prometheus/SNMP senza controllo di rete;

- aggiornare e verificare dipendenze con `pip-audit`;
- provare i modelli reali prima della produzione.

```
bluetoothctl show
.venv/bin/ble-sensors-mqtt --list-plugins
.venv/bin/ble-sensors-mqtt --scan --scan-duration 20 --log-level DEBUG
journalctl -u ble-sensors-mqtt --since today
```

La verifica AGID è un'autovalutazione tecnica, non una certificazione.

13. Build e rollback

`scripts/build-package.sh` genera build UTC YYYYMMDD-HHMM, ZIP, TAR.GZ e SHA-256. Conservare il pacchetto precedente per il rollback e non modificare direttamente `site-packages`.

14. Health di produzione e gate di release

Health e readiness

Quando Prometheus è abilitato, lo stesso server HTTP espone:

- `/metrics` per Prometheus;
- `/healthz` per la liveness del processo;
- `/readyz` per la readiness. Restituisce HTTP 503 fino al primo ciclo riuscito e nuovamente se il polling riuscito diventa troppo vecchio.

L'esposizione non-loopback di Prometheus/health richiede `--allow-external-prometheus`; SNMPv2c non-loopback richiede `--allow-external-snmp`. MQTT remoto o con credenziali in chiaro richiede `--allow-insecure-mqtt`. Questi override sono consensi espliciti al rischio, non default consigliati.

`--plugin-timeout` limita una singola chiamata decoder/cloud (default 15 secondi). `--stale-cycles` stabilisce dopo quanti cicli consecutivi mancanti rimuovere lo stato MQTT retained (default 3). `--state-file` persiste lo stato tra riavvii; l'unità systemd usa `/var/lib/ble-sensors-mqtt/state.json`.

Verifica release

Installare `.[all,dev,release]` ed eseguire:

```
scripts/release-check.sh
```

Il gate richiede lint, test/coverage, Bandit, audit dipendenze, generazione PDF EN/IT, build wheel e sdist, twine check, SBOM CycloneDX e verifica del pacchetto di release. La CI ripete i test runtime su Python 3.11, 3.12 e 3.13.

Per la pubblicazione su GitHub, eseguire prima il push del commit sorgente e poi il push di un tag `vX.Y.Z` corrispondente. Il workflow del tag esegue tutti i gate, costruisce gli artefatti deterministici di release, crea checksum SHA-256 consolidati e pubblica automaticamente la GitHub Release soltanto dopo il successo dei job di test e build. Il job di release verifica che la versione del tag corrisponda a `VERSION` e `pyproject.toml`.